

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1:

Q1. Functional Dependencies & Candidate Keys for STUDENT_COURSE

Relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

This relation stores which student takes which course, along with instructor and grade.

1. Identify All Functional Dependencies (FDs)

We assume the following real-world meanings:

- **SID** → **SName** A student ID uniquely identifies the student name.
- **CID** → **CName, Instructor** A course ID uniquely identifies the course name and instructor.
- **SID, CID** → **Grade** A student's grade depends on both the student and the course.

List of FDs

1. **SID** → **SName**
2. **CID** → **CName, Instructor**
3. **SID, CID** → **Grade**

These are the minimal and realistic dependencies for this schema.

2. Determine Candidate Keys

A **candidate key** must uniquely identify every attribute in the relation.

Let's compute closure of {SID, CID}:

(SID, CID)+ = {SID, CID, SName, CName, Instructor, Grade}

Because:

- SID gives SName
- CID gives CName, Instructor
- SID, CID gives Grade

Thus (SID, CID) determines all attributes → It is a key.

Check if SID alone is a key

SID+ = {SID, SName} → Missing course details → **Not a key**

Check if CID alone is a key

$CID^+ = \{CID, CName, Instructor\} \rightarrow$ Missing student details $\rightarrow$ **Not a key**

Final Candidate Key

{SID, CID}

There is only one candidate key.

3. Sample Table

STUDENT_COURSE Table

SID	SName	CID	CName	Instructor	Grade
S01	Hemalakshmi	C101	DBMS	Dr. Rajkumar	A
S01	Hemalakshmi	C102	AI	Dr. Meena	B+
S02	Arjun	C101	DBMS	Dr. Rajkumar	A-
S03	Priya	C103	Networks	Dr. Kannan	B

Functional Dependencies:

1. $SID \rightarrow SName$
2. $CID \rightarrow CName, Instructor$
3. $SID, CID \rightarrow Grade$

Candidate Key: {SID, CID}

Question 2:

First Normal Form (1NF) requires that:

1. Every attribute contains atomic (single) values.
2. There are no repeating groups or multivalued attributes.
3. Each row can be uniquely identified using a primary key.

Example: Unnormalized Relation

Student_ID	Student_Name	Courses	Marks
101	Ravi	DBMS, Python, Java	85, 90, 78
102	Priya	DBMS, Python	88, 92

Step 1: Identify the repeating groups

For Student 101:

- Courses → DBMS, Python, Java
- Marks → 85, 90, 78

These are repeating groups because one student can have multiple courses and marks.

Step 2: Remove the repeating groups

Create a separate row for each course and its corresponding mark.

Student_ID	Student_Name	Course	Mark
101	Ravi	DBMS	85
101	Ravi	Python	90
101	Ravi	Java	78
102	Priya	DBMS	88
102	Priya	Python	92

Now, every cell contains only **one value**, so all attributes are atomic.

Step 3: Choose the primary key

Student_ID alone cannot be the primary key because one student appears in multiple rows.

Therefore, we can use:

Primary Key = (Student_ID, Course)

This combination uniquely identifies each record.

Final 1NF Relation:

STUDENT_COURSE(Student_ID, Student_Name, Course, Mark)

Primary Key: (Student_ID, Course)

Justification:

The relation is now in **1NF** because:

- Each cell contains a **single atomic value**.
- Repeating/multivalued groups have been eliminated.
- Each record is uniquely identified by **Student_ID + Course**.

Thus, the UNF relation is converted to 1NF by eliminating repeating groups and ensuring atomic values in every attribute.

Question 3:

Given:

Relation: R(A,B,C,D,E)

Functional Dependencies (FDs):

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

We need to decompose the relation into 2NF by eliminating partial dependencies.

Step 1: Find the candidate key

From:

$$A \rightarrow BC$$

and

$$BC \rightarrow D$$

and

$$D \rightarrow E$$

we get:

$$\begin{array}{c} A \\ + \\ = \{A, B, C, D, E\} \end{array}$$

Therefore, A is a candidate key.

Since the candidate key contains only one attribute, there cannot be a partial dependency on a part of the key.

Step 2: Check for partial dependency

A partial dependency occurs when a non-prime attribute depends on only a proper subset of a composite candidate key.

Here, the candidate key is:

A

Since it has only one attribute, there is no proper subset of A.

Therefore, there are no partial dependencies.

Step 3: 2NF decomposition

Since R is already in 2NF, no decomposition is required for 2NF.

R(A,B,C,D,E) is already in 2NF

However, there are transitive dependencies:

$A \rightarrow D \rightarrow E$

So the relation is not in 3NF.

If we continue normalization to 3NF, we can decompose into:

- R1(A,B,C)
- R2(B,C,D)
- R3(D,E)

Final answer

2NF decomposition:

R(A,B,C,D,E)

No decomposition is needed, because the only candidate key is A, which is a single attribute. Thus, partial dependency does not exist.

Question 4:

3NF Decomposition

Given relation:

$R(\text{EmpID}, \text{EmpName}, \text{DeptID}, \text{DeptName}, \text{ManagerID})$

Assume the functional dependencies are:

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Here, EmpID is the candidate key.

Step 1: Identify the transitive dependency

We have:

$\text{EmpID} \rightarrow \text{DeptID}$

and

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore:

$\text{EmpID} \rightarrow \text{DeptName}, \text{ManagerID}$

This is a transitive dependency:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Since DeptName and ManagerID are non-key attributes, this violates 3NF.

Step 2: Decompose the relation

Separate the attributes involved in the transitive dependency.

Relation 1: EMPLOYEE

- $\text{EMPLOYEE}(\text{EmpID}, \text{EmpName}, \text{DeptID})$

- Primary Key: EmpID
- EmpID $\rightarrow$ EmpName, DeptID

Relation 2: DEPARTMENT

DEPARTMENT(DeptID,DeptName,ManagerID)

Primary Key: DeptID

DeptID $\rightarrow$ DeptName, ManagerID

Step 3: Final 3NF relations

EMPLOYEE

EmpID (PK) EmpName DeptID (FK)

DEPARTMENT

DeptID (PK) DeptName ManagerID

Why is this in 3NF?

In EMPLOYEE, EmpID determines all non-key attributes.

In DEPARTMENT, DeptID determines DeptName and ManagerID.

The transitive dependency EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName, ManagerID has been eliminated.

Therefore:

$R \rightarrow$ EMPLOYEE(EmpID,EmpName,DeptID)+DEPARTMENT(DeptID,DeptName,ManagerID)

This decomposition removes the transitive dependency and puts the relation into 3NF.

Question 5:

Given:

R(A,B,C,D)

Functional Dependencies:

- AB $\rightarrow$ C

- $C \rightarrow D$
- $D \rightarrow A$

We need to check BCNF.

Step 1: Find candidate keys

$$AB \rightarrow C$$

Then:

$$C \rightarrow D$$

So:

$$AB^+ = \{A, B, C, D\}$$

Thus, AB is a candidate key.

$$BC \rightarrow$$

From $C \rightarrow D$, and $D \rightarrow A$:

$$BC^+ = \{A, B, C, D\}$$

So BC is also a candidate key.

$$BD \rightarrow$$

From $D \rightarrow A$:

$$BD \rightarrow A$$

Then $AB \rightarrow C$, giving:

$$\begin{aligned} BD &\rightarrow \\ &+ \\ &= \{A, B, C, D\} \end{aligned}$$

So BD is also a candidate key.

Therefore, candidate keys are:

AB, BC, BD

Step 2: Check each FD for BCNF

A relation is in BCNF if, for every non-trivial FD $X \rightarrow Y$, X must be a superkey.

FD 1: $AB \rightarrow C$

AB is a candidate key.

Satisfies BCNF.

FD 2: $C \rightarrow D$

C is not a superkey because:

$C^+ = \{C, D, A\}$

It does not contain B .

Violates BCNF.

FD 3: $D \rightarrow A$

D is also not a superkey.

Violates BCNF.

Therefore:

R is NOT in BCNF

Step 3: Decompose using $C \rightarrow D$

For the violating FD:

$C \rightarrow D$

Decompose R into:

R

1

(C,D)

and

- R2(A,B,C)
- Check R1(C,D)

FD:

$$C \rightarrow D$$

The decomposition eliminates the BCNF violations caused by $C \rightarrow D$ and $D \rightarrow A$.

Question 6:

Experimental Verification of Lossless-Join Decomposition Using Chase Algorithm

Let's use a simple example to demonstrate the tabular (Chase) algorithm.

Given

Relation:

$$R(A,B,C)$$

Functional dependency:

$$A \rightarrow B$$

Decomposition:

$$R1(A,B), R2(A,C)$$

We need to verify whether this decomposition is lossless.

Step 1: Create the initial tableau

Since R has attributes A,B,C, create one row for each decomposed relation.

Use:

$aA, aB, aC \rightarrow$ distinguished symbols

$b_1, b_2, \dots \rightarrow$ nondistinguished symbols

Row	A	B	C
(R_1(A,B))	(a_A)	(a_B)	(b_1)
(R_2(A,C))	(a_A)	(b_2)	(a_C)

For attributes belonging to a particular decomposition, we use the corresponding distinguished symbol a .

Step 2: Apply the FD $A \rightarrow B$

Look for rows having the same value of A .

Both rows have:

$A = a_A$

Therefore, according to:

$A \rightarrow B$

their B values must be made identical.

The first row already has a_B , so replace b_2 with a_B

Row	A	B	C
(R_1)	(a_A)	(a_B)	(b_1)
(R_2)	(a_A)	(a_B)	(a_C)

Step 3: Check for a distinguished row

A decomposition is lossless if, after applying the functional dependencies, a row containing all distinguished symbols appears.

We have:

Row 1 $\rightarrow a_A, a_B, b_1$
Row 2 $\rightarrow a_A, a_B, a_C$

Row 2 contains:

a_A, a_B, a_C

Therefore, the tableau has a distinguished row.

Conclusion

Since a row containing all distinguished symbols is obtained:

The decomposition is LOSSLESS

Thus,

$$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$$

is a lossless-join decomposition with respect to $A \rightarrow B$.

Key rule for Chase Algorithm

If the tableau eventually contains a row consisting entirely of distinguished symbols, the decomposition is lossless. Otherwise, it is lossy.

Question 7:

4NF Decomposition of TEACHER

Given:

TEACHER(TID, Subject, Hobby)

Assume that a teacher can teach **multiple subjects** and have **multiple hobbies**, and these two sets are independent.

Step 1: Identify the Multi-Valued Dependencies

For a teacher, the subjects are independent of the hobbies.

Therefore:

$TID \twoheadrightarrow \text{Subject}$

and equivalently:

$TID \twoheadrightarrow \text{Hobby}$

These are **multi-valued dependencies (MVDs)**.

Step 2: Check for 4NF

A relation is in **4NF** if, for every non-trivial MVD:

$X \twoheadrightarrow Y$

must be a **superkey**.

Here:

But TID is **not a superkey** of the original relation because a teacher can have multiple subjects and hobbies.

Therefore:

TEACHER violates 4NF

Step 3: Decompose the relation

Using the MVD:

$TID \twoheadrightarrow Subject$

decompose into:

Relation 1: TEACHER_SUBJECT

TEACHER_SUBJECT(TID,Subject)

Relation 2: TEACHER_HOBBY

TEACHER_HOBBY(TID,Hobby)

Step 4: Verify 4NF

TEACHER_SUBJECT(TID, Subject)

- $TID \twoheadrightarrow Subject$
- TID is a key of this relation.
- Therefore, it satisfies 4NF.

TEACHER_HOBBY(TID, Hobby)

- $TID \twoheadrightarrow Hobby$
- TID is a key of this relation.
- Therefore, it satisfies 4NF.

Final Answer

$TEACHER(TID, Subject, Hobby) \rightarrow TEACHER_SUBJECT(TID, Subject) + TEACHER_HOBBY(TID, Hobby)$

The decomposition removes the independent multi-valued facts and produces relations in **4NF**.

Question 8:

5NF (Project-Join Normal Form) — Example

Consider the relation:

$R(\text{Supplier}, \text{Part}, \text{Project})$

Suppose a supplier can supply a part, a supplier can work on a project, and a project can use a part. These relationships are **independent**.

Step 1: Identify the Join Dependency

The relation has a **join dependency**:

$\ast(\text{Supplier}, \text{Part}), (\text{Supplier}, \text{Project}), (\text{Part}, \text{Project})$

This means that the original relation can be reconstructed by joining its three projections:

$R = \pi_{SP}(R) \bowtie \pi_{SJ}(R) \bowtie \pi_{PJ}(R)$

where:

- $SP = (\text{Supplier}, \text{Part})$
- $SJ = (\text{Supplier}, \text{Project})$
- $PJ = (\text{Part}, \text{Project})$

Step 2: Show the redundancy

Supplier Part Project

S1	P1	J1
S1	P2	J1
S1	P1	J2
S1	P2	J2

The information is actually composed of three independent relationships:

Supplier–Part

Supplier Part

S1	P1
S1	P2

Supplier–Project

Supplier Project

S1	J1
S1	J2

Part–Project

Part Project

P1	J1
P2	J2

Storing all combinations in one table can cause **redundancy**. For example, if S1 supplies two parts and works on two projects, multiple combinations have to be stored.

Step 3: Decompose into 5NF

Create three relations:

R1(Supplier,Part)
R2(Supplier,Project)
R3(Part,Project)

So:

$R \rightarrow R1 \bowtie R2 \bowtie R3$

Step 4: Verify 5NF

Each decomposed relation represents a single independent relationship.

- SUPPLIER_PART → which parts a supplier supplies
- SUPPLIER_PROJECT → which projects a supplier works on
- PART_PROJECT → which parts are used in which projects

The original relation can be reconstructed by joining these relations.

Therefore, the decomposition eliminates redundancy caused by the join dependency.

Final Answer

$SPJ(\text{Supplier}, \text{Part}, \text{Project}) \rightarrow SP(\text{Supplier}, \text{Part}) + SJ(\text{Supplier}, \text{Project}) + PJ(\text{Part}, \text{Project})$

Thus, the relation is decomposed into **5NF (Project-Join Normal Form)**, where no non-trivial join dependency remains unless it is implied by a candidate key.

Question 9:

Normalizing a Poorly Designed College Database to 3NF

Consider the following poorly designed relation:

COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, FacultyID, FacultyName, Marks)

Assume the functional dependencies are:

- StudentID $\rightarrow$ StudentName, DeptID
- DeptID $\rightarrow$ DeptName
- CourseID $\rightarrow$ CourseName, FacultyID
- FacultyID $\rightarrow$ FacultyName
- StudentID, CourseID $\rightarrow$ Marks

The candidate key is:

(StudentID, CourseID)

1. Identify the anomalies

A. Insertion Anomaly

Suppose a **new course** is introduced but no student has enrolled yet.

We cannot insert the course information because StudentID and Marks are part of the same table.

Similarly, adding a new department requires a student record.

Problem: We cannot store independent information without unrelated information.

B. Update Anomaly

Suppose the name of department D01 changes from **Computer Science** to **Computer Science and Engineering**.

The department name may appear in hundreds of student records.

We must update every occurrence.
 If even one row is missed, inconsistent data occurs.
Problem: The same information is stored repeatedly.

C. Deletion Anomaly

Suppose the only student enrolled in course C101 leaves the college.
 Deleting that student's record also deletes the only information about:

- Course C101
- Course name
- Faculty teaching it

Problem: Deleting one fact unintentionally removes another fact.

2. Convert to 1NF

Ensure every attribute contains only atomic values.

StudentID	StudentName	DeptID	DeptName	CourseID	CourseName	FacultyID	FacultyName	Marks
S01	Ravi	D01	CSE	C101	DBMS	F01	Kumar	85
S01	Ravi	D01	CSE	C102	Python	F02	Priya	90
S02	Anu	D02	ECE	C101	DBMS	F01	Kumar	78

3. Convert to 2NF

The composite key is:

(StudentID, CourseID)

But there are **partial dependencies**:

StudentID → StudentName, DeptID

And

CourseID → CourseName, FacultyID

These attributes depend on only part of the composite key.

Therefore, decompose into:

STUDENT

STUDENT(StudentID, StudentName, DeptID)

COURSE

COURSE(CourseID, CourseName, FacultyID)

ENROLLMENT

ENROLLMENT(StudentID,CourseID,Marks)

4. Convert to 3NF

There are still transitive dependencies:

$$\text{StudentID} \rightarrow \text{DeptID}$$

and

$$\text{DeptID} \rightarrow \text{DeptName}$$

Therefore:

$$\text{StudentID} \rightarrow \text{DeptName}$$

Similarly:

$$\text{CourseID} \rightarrow \text{FacultyID}$$

and

$$\text{FacultyID} \rightarrow \text{FacultyName}$$

Therefore:

$$\text{CourseID} \rightarrow \text{FacultyName}$$

These violate 3NF.

So, decompose further.

STUDENT

STUDENT(StudentID,StudentName,DeptID)

DEPARTMENT

DEPARTMENT(DeptID,DeptName)

COURSE

COURSE(CourseID,CourseName,FacultyID)

FACULTY
FACULTY(FacultyID, FacultyName)

ENROLLMENT
ENROLLMENT(StudentID, CourseID, Marks)

Final 3NF Schema

Relation	Attributes	Primary Key
STUDENT	StudentID, StudentName, DeptID	StudentID
DEPARTMENT	DeptID, DeptName	DeptID
COURSE	CourseID, CourseName, FacultyID	CourseID
FACULTY	FacultyID, FacultyName	FacultyID
ENROLLMENT	StudentID, CourseID, Marks	StudentID + CourseID

Final conclusion

The original college relation suffers from **insertion, deletion, and update anomalies** because unrelated facts are stored together and data is repeated.

Question 10:

Consider:

$R(A, B, C, D, E)$

with functional dependencies:

$F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow E, E \rightarrow A\}$

We need to find the closure of attribute sets and determine all candidate keys.

1. Find

Start with:

$$A^+ = \{A\}$$

Using $A \rightarrow B$:

$$A^+ = \{A, B\}$$

Using $B \rightarrow C$:

$$A^+ = \{A, B, C\}$$

We cannot use $CD \rightarrow E$ because we don't have D.

Therefore:

$$A^+=\{A,B,C\}$$

So **A is not a key.**

2. Find D^+

Start:

$$D^+=\{D\}$$

There is no FD with only D on the left.

$$D^+=\{D\}$$

So D is not a key.

3. Find E^+

Start:

$$E^+=\{E\}$$

Using:

$$E \rightarrow A$$

we get:

$$\{E,A\}$$

Then:

$$A \rightarrow B$$

gives:

$$\{E,A,B\}$$

Then:

$$B \rightarrow C$$

gives:

$$E^+=\{A,B,C,E\}$$

D is missing, so E is not a key.

4. Find $(AD)^+$

Start:

$$(AD)^+=\{A,D\}$$

Using $A \rightarrow B$:

$$\{A,B,D\}$$

Using $B \rightarrow C$:

$$\{A,B,C,D\}$$

Now we have C and D, so:

$$CD \rightarrow E$$

Therefore:

$$(AD)^+=\{A,B,C,D,E\}$$

Thus AD is a superkey.

Check minimality:

- $A^+ \neq R$
- $D^+ \neq R$

Therefore:

AD is a candidate key

5. Find $(DE)^+$

Start:

$$(DE)^+=\{D,E\}$$

Using $E \rightarrow A$:

$$\{D,E,A\}$$

Using $A \rightarrow B$:

$$\{D,E,A,B\}$$

Using $B \rightarrow C$:

$$\{D,E,A,B,C\}$$

Therefore:

$$(DE)^+=\{A,B,C,D,E\}$$

So DE is a superkey.

Neither D nor E alone is a key, therefore:

DE is a candidate key

6. Are there any other candidate keys?

Notice that D never appears on the right-hand side of any FD.

Therefore, every candidate key must contain D.

To obtain the remaining attributes:

- $D + A \text{ works} \rightarrow AD$
- $D + E \text{ works} \rightarrow DE$

Other combinations such as BD or CD do not determine all attributes.

Hence, all candidate keys are:

$$\text{Candidate Keys}=\{AD, DE\}$$

Attribute set	Closure	Key?
(A)	(A,B,C)	No
(D)	(D)	No
(E)	(A,B,C,E)	No
(AD)	(A,B,C,D,E)	Candidate Key
(DE)	(A,B,C,D,E)	Candidate Key

Therefore, the candidate keys of R are:

AD, DE
